

Содержание

Введение	5
1 Исследовательский раздел	7
1.1 Анализ предметной области	8
1.2 Постановка цели и задач	10
1.3 Анализ существующих аналогов.....	11
1.3.1 Wordle	12
1.3.2 Spelling Bee	13
1.3.3 Tradle	15
1.4 Инструментальные средства разработки	16
1.5 Вывод	17
2 Специальный раздел	19
2.1 Пользовательский сценарий взаимодействия	19
2.2 Выбор и описание инструментальных средств разработки	21
2.2.1 Язык программирования C#.....	25
2.2.2 Среда разработки Microsoft Visual Studio	27
2.2.3 Работа с базой данных SQL.....	28
2.2.4 Пользовательский интерфейс приложения	30
2.2.5 Архитектура приложения.....	31
2.4 Вывод	33
3 Программная реализация	34
3.1 Реализация интерфейса пользователя	35
3.2 Разработка программного продукта	38
Заключение	40
Перечень использованных информационных ресурсов.....	42

					РПМ.340000.000 ПЗ						
Изм.	Лист	№ докум.	Подпись	Дата	Эрудит тренажёр			Лит.	Лист	Листов	
Разраб.		Дацук Д.А									
Провер.		Запорожец О.И.								4	42
Реценз.								ТИ (филиал) ДГТУ в г.Азове Кафедра «ВТуп»			
Н. Контр.											
Утверд.		Чумак. И.В.									

Введение

В условиях активной цифровизации бизнеса и стремительного развития информационных технологий программное обеспечение становится неотъемлемой частью деятельности практически любой организации.

Особенно актуально внедрение специализированных информационных систем в сфере торговли и услуг, где критически важны скорость обслуживания клиентов, точность учета данных и удобство взаимодействия между сотрудниками и покупателями.

Одной из таких динамично развивающихся сфер является автомобильный бизнес, а именно деятельность автосалонов, которая требует обработки огромных массивов данных о технических характеристиках моделей, складских запасах и истории взаимодействия с контрагентами.

В современных условиях разработка специализированного программного приложения для автосалона становится актуальной и востребованной задачей, так как позволяет автоматизировать ключевые бизнес-процессы, снизить вероятность ошибок из-за человеческого фактора, повысить качество обслуживания и улучшить общую управляемость предприятия.

Кроме того, наличие удобного и функционального программного решения способствует росту конкурентоспособности компании на рынке.

Целью данной курсовой работы является проектирование и разработка приложения для автосалона, ориентированного на автоматизацию основных процессов продаж и взаимодействия с клиентами.

Для достижения этой цели в рамках работы проводится анализ предметной области, формулируются цели и задачи разработки, а также осуществляется проектирование пользовательских сценариев взаимодействия с системой и архитектуры базы данных. Основное внимание уделяется

созданию инструмента, который позволит эффективно управлять клиентской базой и оптимизировать рабочий цикл сотрудников отдела продаж.

Практическая значимость работы заключается в возможности использования разработанного приложения или его концепции в реальной деятельности автосалона, а также в приобретении профессиональных навыков анализа бизнес-процессов и проектирования современных информационных систем.

Результаты исследования могут послужить базой для дальнейшего развития и масштабирования цифровой инфраструктуры предприятия автомобильной отрасли.

1 Исследовательский раздел

Исследовательский раздел данной работы посвящен глубокому анализу предметной области и изучению теоретических основ создания игровых обучающих систем, на примере разработки приложения «Эрудит-тренажер».

В эпоху повсеместной цифровизации образования концепция геймификации обучения приобретает особую значимость, превращая рутинный процесс расширения словарного запаса в увлекательную интеллектуальную игру.

В основе проектируемого тренажера лежит популярная во всем мире механика лингвистических головоломок, где игроку необходимо за ограниченное количество попыток угадать секретное слово, используя систему цветowych или символьных подсказок.

Исследование начинается с детального изучения психологии игрового процесса и механизмов удержания внимания, так как успех подобного приложения напрямую зависит от баланса между сложностью задач и достижимостью результата.

Важной частью раздела является системный анализ существующих программных решений, таких как оригинальный проект Wordle и его многочисленные модификации, что позволяет выделить наиболее эффективные подходы к реализации игрового цикла и выявить пробелы в функциональности, которые можно устранить в рамках данной курсовой работы.

Техническая составляющая исследования охватывает вопросы формирования и обработки лексических массивов данных.

Проектирование качественного тренажера слов требует не просто наличия словаря, но и разработки алгоритмов фильтрации слов по длине, исключения редких или узкоспециализированных терминов, а также обеспечения высокой скорости поиска по базе данных при каждой проверке пользовательского ввода.

					РПМ.340000.000 ПЗ	Лист.
						7
Изм.	Лист	№ докум.	Подп.	Дата		

Особое внимание уделяется анализу алгоритмической сложности процесса сопоставления букв, который должен корректно обрабатывать повторяющиеся символы и мгновенно выдавать результат, сохраняя отзывчивость интерфейса.

Кроме того, в данном разделе рассматриваются требования к кроссплатформенности и адаптивности графической оболочки, так как современные пользователи ожидают корректного отображения игровых элементов на устройствах с различными экранными разрешениями.

Изучение предметной области также затрагивает вопросы хранения локальной статистики игрока, что необходимо для создания элемента соревновательности и отслеживания прогресса обучения.

Таким образом, исследовательский этап закладывает комплексный теоретический фундамент, объединяющий лингвистические правила, психологические принципы проектирования интерфейсов и современные стандарты разработки программного обеспечения, что позволяет перейти к непосредственному проектированию архитектуры и реализации функционала приложения.

1.1 Анализ предметной области

Анализ предметной области является критически важным этапом проектирования, в ходе которого определяются ключевые правила, логические ограничения и функциональные границы будущего приложения «Эрудит-тренажер».

Исследование показывает, что программный продукт относится к категории лингвистических логических игр, где основной задачей пользователя является идентификация загаданного слова за фиксированное количество попыток, обычно равное шести.

Предметная область характеризуется строгими правилами комбинаторики и лингвистики: каждое вводимое слово должно существовать

					РПМ.340000.000 ПЗ	Лист.
						8
Изм.	Лист	№ докум.	Подп.	Дата		

в словаре используемого языка и соответствовать заданной длине, которая в классической реализации составляет пять символов. Важнейшим аспектом анализа является механика обратной связи, основанная на изменении визуального состояния игровых ячеек.

Система должна анализировать каждое введенное слово и сравнивать его с эталоном, разделяя символы на три категории: буквы, находящиеся на своих позициях; буквы, присутствующие в слове, но занимающие иные позиции; и символы, полностью отсутствующие в целевой лексической единице.

Этот алгоритм требует особого внимания при проектировании, так как необходимо корректно обрабатывать случаи с повторяющимися буквами, чтобы не дезинформировать игрока и сохранять логическую целостность процесса.

В ходе анализа также выявляется необходимость интеграции обширной базы имен существительных в именительном падеже и единственном числе, что накладывает определенные требования к подсистеме хранения данных. Важной частью предметной области выступает психологический аспект тренировки памяти и аналитического мышления: приложение должно не просто развлекать, но и выступать в роли тренажера, позволяющего пользователю расширять активный словарный запас и улучшать навыки правописания.

С точки зрения взаимодействия пользователя с системой, предметная область охватывает процесс управления через виртуальную или физическую клавиатуру, где каждое действие должно сопровождаться мгновенной валидацией на соответствие правилам русского (или иного выбранного) языка. Кроме того, рассматривается контекст завершения игровой сессии, который предполагает два сценария — победу при полном совпадении слова или поражение при исчерпании доступных попыток, что требует реализации механизмов подведения итогов и отображения правильного ответа. Таким

					<i>РПМ.340000.000 ПЗ</i>	Лист.
						9
Изм.	Лист	№ докум.	Подп.	Дата		

образом, детальный разбор предметной области позволяет четко формализовать требования к программной логике, структуре данных и интерфейсным решениям, обеспечивая создание качественного интеллектуального тренажера.

1.2 Постановка цели и задач

В рамках подраздела, посвященного постановке цели и задач, осуществляется четкая формулировка планируемого результата работы и определение этапов его достижения.

Основной целью курсовой работы является проектирование и программная реализация интерактивного игрового приложения «Эрудит-тренажер» на языке высокого уровня, которое позволит пользователям эффективно развивать логическое мышление и лингвистические навыки в цифровой среде.

Данная цель предполагает создание не просто развлекательного продукта, а надежного инструмента с интуитивно понятным интерфейсом и выверенной игровой логикой, способного функционировать в соответствии с заданными правилами и ограничениями.

Для достижения поставленной цели необходимо решить ряд последовательных задач, первой из которых является детальный анализ требований к функциональности системы, включающий описание всех возможных игровых ситуаций и состояний.

Далее следует этап проектирования архитектуры приложения, где определяется структура хранения словарного запаса и разрабатывается логическая модель проверки вводимых комбинаций символов.

Одной из ключевых задач выступает выбор и обоснование инструментальных средств разработки, обеспечивающих создание стабильного и производительного кода, а также проектирование интерфейса

пользователя, который должен быть эргономичным и визуально информативным.

Непосредственная программная реализация предполагает написание чистого и документированного кода для обработки игровых сессий, управления попытками пользователя и реализации системы подсказок. Заключительной задачей является проведение тестирования готового продукта для выявления и устранения возможных ошибок в алгоритмах сравнения слов, а также проверка корректности работы приложения на различных наборах данных.

Комплексное выполнение указанных задач позволит создать законченное программное решение, полностью соответствующее современным требованиям к образовательным тренажерам и интеллектуальным играм.

1.3 Анализ существующих аналогов

В рамках подраздела, посвященного анализу существующих аналогов, рассматривается современный рынок программных решений в сегменте лингвистических головоломок и интеллектуальных тренажеров.

На сегодняшний день существует широкий спектр приложений, использующих схожие игровые механики, начиная от всемирно известных веб-проектов и заканчивая мобильными приложениями, адаптированными под различные платформы и языковые группы.

Изучение данных программных продуктов позволяет выявить наиболее удачные подходы к реализации пользовательского интерфейса, способов визуализации подсказок и методов формирования словарных баз.

В качестве основных объектов для сравнения выделяются такие проекты, как оригинальная англоязычная игра Wordle, ставшая эталоном жанра, её популярные русскоязычные адаптации, а также более комплексные

					РПМ.340000.000 ПЗ	Лист.
						11
Изм.	Лист	№ докум.	Подп.	Дата		

мобильные тренажеры слов, предлагающие расширенные возможности статистики и обучения.

Краткий обзор данных систем показывает, что, несмотря на общую игровую концепцию, каждый аналог имеет свои уникальные особенности в архитектуре и функциональном наполнении.

Более подробный технический и функциональный анализ выбранных программных продуктов, их сравнительные характеристики, а также сильные и слабые стороны будут детально рассмотрены в последующих разделах работы, что позволит обосновать необходимость разработки собственного приложения и выделить его конкурентные преимущества.

1.3.1 Wordle

В данном подразделе рассматривается наиболее значимый аналог и первоисточник популярности современных лингвистических игр — проект Wordle. Игра была разработана британским программистом Джошем Уордлом в 2021 году и изначально создавалась как персональный подарок, однако в кратчайшие сроки приобрела статус глобального цифрового феномена.

Основная концепция Wordle заключается в ежедневном отгадывании секретного слова, состоящего из пяти букв, за фиксированное количество попыток, равное шести. Характерной особенностью оригинала является принцип «одна игра в день», что создает эффект дефицита контента и способствует формированию устойчивой привычки у пользователей. Логотип Wordle показана на рисунке 1



Рисунок 1 – Логотип Wordle

Механика игры построена на цветовой индикации, которая стала стандартом для всех последующих адаптаций: после ввода слова буквы окрашиваются в серый, желтый или зеленый цвет в зависимости от их наличия в загаданном слове и правильности позиции.

Популярность Wordle обусловлена не только простотой правил, но и уникальной системой обмена результатами в социальных сетях, где пользователи делятся сеткой цветных квадратов, не раскрывая само слово.

В 2022 году проект был приобретен изданием The New York Times, что подтвердило высокую коммерческую и культурную ценность данной игровой модели.

Анализ Wordle позволяет выделить ключевые функциональные элементы, такие как управление словарем наиболее употребительных слов, алгоритм мгновенной проверки ввода и минималистичный дизайн, которые легли в основу проектируемого в данной курсовой работе «Эрудит-тренажера».

Изучение этого аналога дает возможность перенять успешный опыт реализации игровой логики, одновременно адаптируя её под специфику русскоязычного словаря и добавляя элементы тренажера для более глубокого обучения.

1.3.2 Spelling Bee

В качестве второго значимого аналога в рамках исследования рассматривается игра Spelling Bee, которая, как и Wordle, получила широкое признание благодаря платформе The New York Times.

Данный проект представляет собой интеллектуальную головоломку, основанную на составлении максимально возможного количества слов из фиксированного набора из семи букв, расположенных в форме пчелиных сот.

					РПМ.340000.000 ПЗ	Лист.
						13
Изм.	Лист	№ докум.	Подп.	Дата		

Ключевым правилом механики является обязательное использование центральной буквы в каждом предложенном слове, при этом сами слова должны состоять минимум из четырех символов.

В отличие от линейного процесса угадывания в Wordle, Spelling Bee ориентирована на объемное знание лексики и умение находить сложные комбинации внутри ограниченного набора знаков.

Логотип Spelling Bee показан на рисунке 2

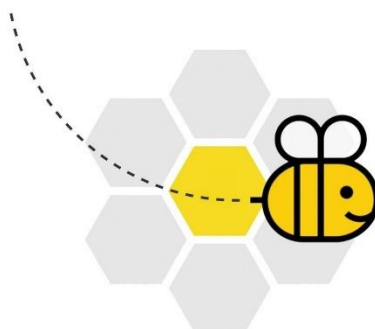


Рисунок 2 – Логотип Spelling Bee

Анализ данного приложения позволяет выделить несколько важных аспектов, полезных для разработки «Эрудит-тренажера».

Во-первых, это система ранжирования и начисления очков в зависимости от длины слова и использования редких букв, что превращает игру в полноценный инструмент для измерения когнитивных навыков.

Во-вторых, в Spelling Bee реализована концепция «панграмм» — слов, включающих в себя все семь доступных букв, что дает игроку дополнительную мотивацию и вносит элемент азарта в процесс обучения.

С точки зрения программной реализации, данный аналог демонстрирует важность качественной интеграции со словарями, так как система должна мгновенно проверять вводимые варианты на соответствие нормам языка, отсекая сленг или несуществующие формы.

Изучение Spelling Bee подчеркивает необходимость внедрения в

проектируемый тренажер элементов прогрессии и оценки сложности, что позволяет пользователю не только угадывать слова, но и наглядно видеть рост своего словарного запаса и интеллектуальных способностей.

Таким образом, опыт реализации Spelling Bee дополняет исследование методами вовлечения игрока через накопление баллов и достижение определенных статусных уровней внутри игровой сессии.

1.3.3 Tradle

В качестве еще одного инновационного аналога, демонстрирующего возможности адаптации игровой механики Wordle к другим предметным областям, рассматривается проект Tradle.

Данное веб-приложение переносит классический принцип логического поиска в сферу мировой экономики и международной торговли.

Вместо угадывания букв пользователю предлагается идентифицировать страну на основе данных о её экспорте, представленных в виде наглядной древовидной карты секторов экономики.

Визуализация данных в Tradle базируется на сложности и структуре товаров, которые страна поставяляет на мировой рынок, что делает игру не просто развлекательной, но и глубоко образовательной в контексте экономической географии.

Логотип Tradle показан на рисунке



Рисунок 3 – Логотип Tradle

Анализ Tradle позволяет выделить уникальные подходы к системе подсказок и обратной связи, которые могут быть полезны при проектировании «Эрудит-тренажера».

После каждой неудачной попытки система сообщает пользователю не только степень близости его ответа к цели, но и указывает направление и расстояние в километрах до искомой страны, используя географические координаты.

Это демонстрирует, как игровые алгоритмы могут успешно оперировать не только текстовыми строками, но и числовыми данными, преобразуя их в понятные пользователю ориентиры.

С точки зрения программной реализации, Tradle интересен как пример интеграции приложения с внешними аналитическими базами данных, такими как Обсерватория экономической сложности.

Изучение этого аналога подтверждает универсальность заложенной в курсовой работе механики и показывает, что добавление контекстных подсказок и визуализации данных значительно расширяет функциональный потенциал приложения, превращая его из простой игры в комплексный инструмент для изучения закономерностей в выбранной области знаний.

1.4 Инструментальные средства разработки

В рамках данного подраздела рассматривается обоснование выбора программного обеспечения и языков программирования, необходимых для реализации проекта «Эрудит-тренажер».

Основным инструментом разработки была выбрана интегрированная среда разработки Microsoft Visual Studio, которая является признанным стандартом в индустрии для создания приложений на платформе .NET.

Выбор данной среды обусловлен наличием мощных средств отладки, встроенных инструментов для проектирования графических интерфейсов и

высокой степени автоматизации процесса написания кода.

Visual Studio предоставляет разработчику обширный инструментарий для управления зависимостями и ресурсами проекта, что критически важно при работе со словарями и графическими ассетами игрового приложения.

Использование этой среды позволяет значительно сократить время на рутинные операции по настройке проекта и сосредоточиться на реализации сложной алгоритмической части тренажера.

В качестве основного языка программирования выбран C# — современный объектно-ориентированный язык, сочетающий в себе высокую производительность и выразительность синтаксиса.

Применение C# для разработки «Эрудита» оправдано его мощными возможностями по работе со строковыми данными и коллекциями, что является фундаментом для игровых механик типа Wordle.

Наличие встроенных библиотек .NET позволяет эффективно реализовывать логику сравнения слов, управлять игровыми состояниями и обеспечивать быструю валидацию пользовательского ввода по базе данных. Кроме того, строго типизированная природа языка минимизирует количество логических ошибок на этапе компиляции, обеспечивая стабильную работу приложения.

Таким образом, сочетание Visual Studio и языка C# формирует надежный технологический фундамент, позволяющий создать качественное, производительное и легко поддерживаемое программное обеспечение, полностью отвечающее целям курсовой работы

1.5 Вывод

Завершающим этапом первого раздела является обобщение результатов проведенного исследования, которое служит логическим переходом к практическому этапу проектирования системы.

					РПМ.340000.000 ПЗ	Лист.
						17
Изм.	Лист	№ докум.	Подп.	Дата		

В ходе выполнения исследовательского раздела был проведен комплексный анализ предметной области, позволивший четко определить механику функционирования интеллектуального приложения «Эрудит-тренажер» и формализовать правила взаимодействия пользователя с игровой средой.

Изучение теоретических основ геймификации и лингвистических тренажеров подтвердило актуальность разработки собственного программного продукта, способствующего развитию когнитивных навыков и расширению словарного запаса.

Подробный разбор существующих аналогов, таких как Wordle, Spelling Bee и Tradle, позволил выделить наиболее успешные рыночные практики и определить вектор развития собственного проекта, направленный на создание удобного интерфейса и надежной системы валидации данных.

Обоснование выбора инструментальных средств разработки в пользу языка программирования C# и среды Microsoft Visual Studio подтвердило готовность технической базы для реализации сложных алгоритмов сопоставления символов и управления игровыми сессиями.

Таким образом, сформированный в первой главе теоретический фундамент и четкая постановка задач позволяют перейти к следующему этапу курсовой работы — непосредственному проектированию архитектуры приложения, разработке структуры базы данных и формированию пользовательских сценариев, которые обеспечат эффективное функционирование итогового программного решения.

2 Специальный раздел

В рамках специального раздела осуществляется переход от концептуального описания тренажера слов к его техническому проектированию и программной реализации в среде Visual Studio.

Данный этап является ключевым, так как именно здесь закладывается логическая структура приложения, определяются методы обработки данных и описывается механика взаимодействия внутренних модулей системы.

Специальный раздел охватывает вопросы архитектурного планирования, детальную проработку алгоритмов сравнения строк, организацию системы хранения словаря, а также проектирование пользовательского интерфейса.

Основное внимание уделяется обеспечению отказоустойчивости программы, корректности выполнения игровых циклов и оптимизации ресурсов при работе с лексическими массивами.

В процессе проектирования учитываются принципы объектно-ориентированного программирования, что позволяет создать гибкую структуру кода, удобную для тестирования и последующего масштабирования функционала.

Результатом данного раздела станет полное техническое описание системы, готовое к этапу отладки и внедрения, что позволит реализовать проект «Эрудит-тренажер» как полноценный и стабильный программный продукт, отвечающий всем требованиям качества и удобства использования.

2.1 Пользовательский сценарий взаимодействия

Проектирование пользовательского сценария взаимодействия является фундаментом для разработки интерфейса и программной логики приложения «Эрудит-тренажер».

					РПМ.340000.000 ПЗ	Лист.
						19
Изм.	Лист	№ докум.	Подп.	Дата		

Сценарий начинается с момента запуска программы, когда пользователь видит главное игровое окно, содержащее пустую сетку для ввода букв и виртуальную клавиатуру.

Первым этапом взаимодействия является инициация игровой сессии, при которой система в фоновом режиме случайным образом выбирает секретное слово из предустановленного словаря.

Пользователь приступает к вводу первого варианта слова, используя физическую клавиатуру или графические элементы интерфейса.

В процессе ввода система должна обеспечивать визуальное отображение набираемых символов в активной строке сетки, позволяя игроку корректировать ввод до момента окончательного подтверждения попытки.

После нажатия клавиши подтверждения (Enter) сценарий переходит к фазе валидации и обработки данных.

Система проверяет введенную последовательность символов на соответствие двум критериям: полнота слова (заполнение всех ячеек строки) и наличие данного слова в лексической базе данных.

Если слово не проходит проверку, сценарий предусматривает вывод уведомления, информирующего пользователя об ошибке, при этом попытка не засчитывается, и состояние игрового поля не меняется.

В случае успешной валидации запускается алгоритм сравнения, который мгновенно меняет цветовое состояние ячеек и соответствующих клавиш на виртуальной клавиатуре.

Это критически важный момент взаимодействия, так как пользователь получает интеллектуальную подсказку для формирования следующей догадки.

Дальнейший сценарий носит циклический характер: пользователь анализирует полученную цветовую индикацию и предпринимает последующие попытки, количество которых ограничено шестью итерациями. Взаимодействие завершается при наступлении одного из двух финальных

					<i>РПМ.340000.000 ПЗ</i>	Лист.
						20
Изм.	Лист	№ докум.	Подп.	Дата		

событий.

Первое — сценарий победы, когда пользователь успешно угадывает слово, после чего система блокирует дальнейший ввод, выводит поздравительное сообщение и обновляет статистику достижений.

Второе — сценарий поражения, наступающий при исчерпании всех попыток без нахождения верного ответа, при котором пользователю открывается загаданное слово.

Завершение сценария предполагает возможность перезапуска игры, что возвращает систему в исходное состояние с выбором новой лексической единицы.

Таким образом, спроектированный сценарий обеспечивает непрерывный и интуитивно понятный цикл взаимодействия, минимизируя когнитивную нагрузку на пользователя и фокусируя его внимание на решении лингвистической задачи.

2.2 Выбор и описание инструментальных средств разработки

В данном подразделе подробно обосновывается выбор технологического стека, использованного для реализации программного продукта «Эрудит-тренажер».

Основным инструментом разработки выступает интегрированная среда (IDE) Microsoft Visual Studio, которая предоставляет комплексный набор средств для написания, отладки и компиляции кода на платформе .NET.

Выбор данной среды обусловлен её развитой экосистемой, наличием визуального дизайнера форм, позволяющего оперативно проектировать пользовательский интерфейс, и мощными инструментами статического анализа кода, что критически важно для минимизации ошибок в логических блоках игры.

Visual Studio обеспечивает бесшовную интеграцию с системами

					РПМ.340000.000 ПЗ	Лист.
						21
Изм.	Лист	№ докум.	Подп.	Дата		

контроля версий и предоставляет удобные механизмы управления ресурсами, такими как внешние текстовые файлы со словарями и графические элементы оформления.

На рисунке 4 показана форма авторизации пользователя

Рисунок 4 – Форма авторизации

В качестве языка программирования выбран C#, который является объектно-ориентированным языком с сильной типизацией, идеально подходящим для создания десктопных приложений с четкой логической структурой.

Выбор C# продиктован необходимостью реализации сложных алгоритмов обработки строковых данных, таких как сравнение введенного пользователем слова с эталоном и распределение цветowych подсказок.

Язык обладает богатой стандартной библиотекой классов, включая

пространство имен System.Collections.Generic, которое используется для эффективного хранения и поиска слов в динамических массивах и списках.

Использование платформы .NET (Windows Forms или WPF) позволяет реализовать событийно-ориентированную модель управления, где каждое действие пользователя — нажатие клавиши или клик по кнопке — мгновенно обрабатывается соответствующим методом.

На рисунке 5 показана форма регистрации

Рисунок 5 – Форма регистрации

Технический выбор также затронул формат хранения данных: для работы со словарным запасом было решено использовать текстовые файлы формата .txt или ресурсы внутри сборки, что обеспечивает высокую скорость загрузки и простоту редактирования базы слов без необходимости развертывания тяжеловесных систем управления базами данных.

На рисунке 6 показан сам эрудит

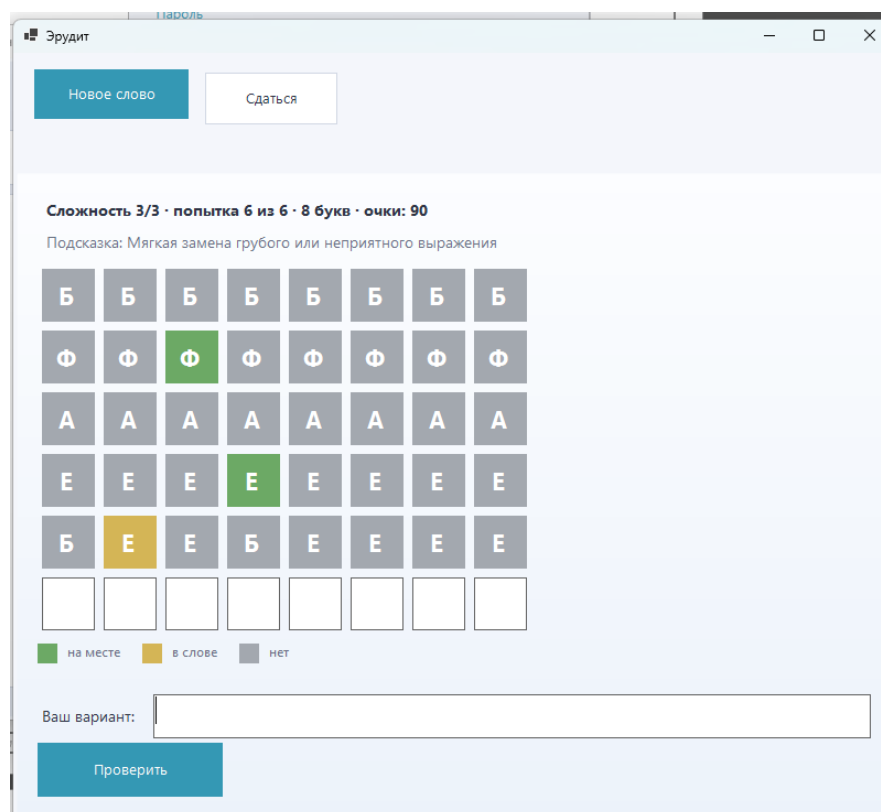


Рисунок 6 – Главная форма игры

Такой подход делает приложение легковесным и переносимым.

На рисунке показана главная форма программы

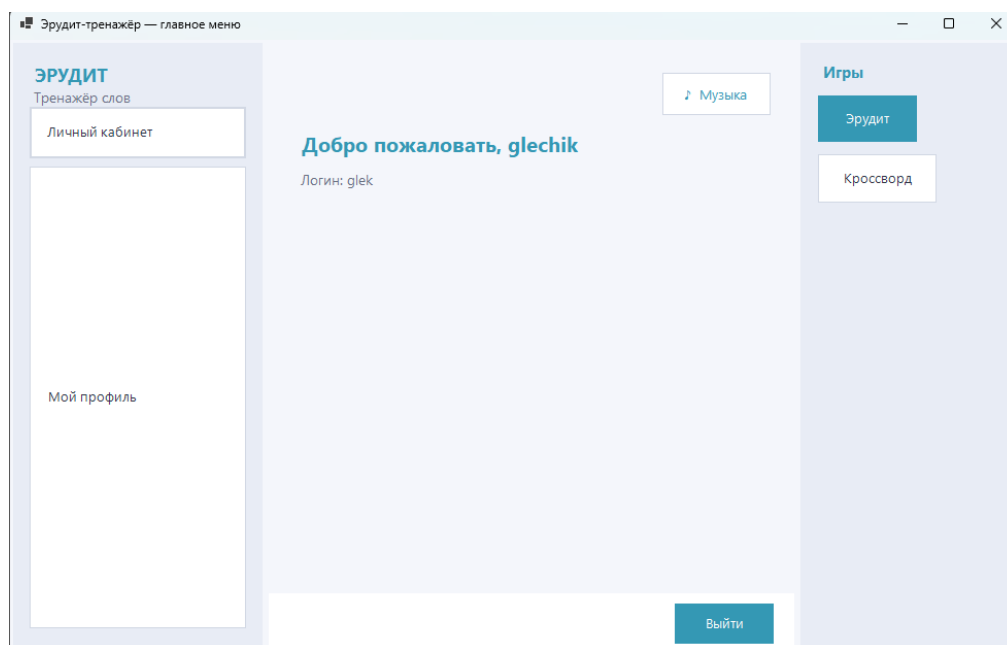


Рисунок 6 – Главная форма программы

Совокупность выбранных инструментальных средств позволяет создать надежное приложение, обладающее высокой производительностью и соответствующее современным стандартам разработки программного обеспечения, обеспечивая при этом простоту дальнейшего сопровождения и модификации кода.

2.2.1 Язык программирования C#

В данном подразделе подробно рассматриваются технические характеристики и преимущества выбранного языка программирования. C# представляет собой современный высокоуровневый объектно-ориентированный язык, разработанный корпорацией Microsoft специально для платформы .NET.

Логотип C# показан на рисунке 7.



Рисунок 7 – Логотип C#

Его выбор в качестве основного средства реализации «Эрудит-

тренажера» обусловлен сочетанием высокой производительности, безопасности и выразительности синтаксиса.

Язык обладает строгой типизацией, что позволяет выявлять значительную часть программных ошибок еще на этапе компиляции, обеспечивая тем самым надежность игровых алгоритмов.

В контексте разработки лингвистической игры ключевую роль играют мощные встроенные средства C# для работы со строками и текстовой информацией.

Поскольку механика игры строится на постоянном сравнении массивов символов и поиске подстрок, стандартные методы класса String позволяют реализовать логику проверки слов с минимальными затратами ресурсов и высокой скоростью выполнения.

Архитектура языка C# поддерживает принципы инкапсуляции, наследования и полиморфизма, что позволяет структурировать код приложения в виде независимых и взаимодействующих классов. Например, логику игрового сеанса, управление словарем и обработку пользовательского интерфейса можно вынести в отдельные модули, что упрощает отладку и модификацию программы.

Важным техническим аспектом является наличие автоматического управления памятью (Garbage Collection), которое освобождает разработчика от необходимости вручную распределять ресурсы, предотвращая утечки памяти при длительных игровых сессиях.

Кроме того, поддержка современных конструкций, таких как коллекции (List, Dictionary) и язык интегрированных запросов LINQ, позволяет эффективно манипулировать базой слов, осуществлять быструю фильтрацию по длине и проводить проверку на существование лексемы в словаре.

Таким образом, язык C# предоставляет весь необходимый инструментарий для создания гибкого, стабильного и расширяемого программного продукта, позволяя эффективно реализовать как внутреннюю

					РПМ.340000.000 ПЗ	Лист.
Изм.	Лист	№ докум.	Подп.	Дата		26

математическую логику игры, так и её внешнее визуальное представление.

2.2.2 Среда разработки Microsoft Visual Studio

Для реализации программного проекта «Эрудит-тренажер» была выбрана интегрированная среда разработки (IDE) Microsoft Visual Studio, которая является одним из наиболее мощных и универсальных инструментов для создания приложений на платформе .NET.

Выбор данной среды обусловлен наличием комплексного инструментария, охватывающего весь цикл жизненного цикла разработки ПО — от проектирования пользовательского интерфейса до глубокой отладки и профилирования кода.

На рисунке 8 показан логотип VS



Рисунок 8 – Логотип Visual Studio

Одной из ключевых особенностей Visual Studio является встроенный визуальный конструктор (Designer), который позволяет проектировать графическую оболочку игры в интерактивном режиме. Это значительно упрощает процесс размещения игровых ячеек, кнопок управления и клавиатуры, позволяя разработчику мгновенно видеть результат компоновки и настраивать свойства объектов через удобную панель инструментов.

Техническая значимость использования Visual Studio также заключается

в наличии интеллектуальной системы подсказок IntelliSense, которая ускоряет процесс написания кода на языке C#, предлагая подходящие методы и структуры данных в режиме реального времени.

В контексте разработки сложной игровой логики, связанной со сравнением слов, критически важными становятся инструменты отладки Visual Studio.

Среда позволяет устанавливать точки останова, отслеживать значения переменных на каждом шаге выполнения алгоритма и анализировать состояние стека вызовов, что гарантирует корректность работы системы цветовой индикации букв.

Кроме того, IDE предоставляет удобные средства для управления ресурсами проекта: через обозреватель решений (Solution Explorer) осуществляется интеграция текстовых файлов со словарями, иконок и шрифтов, которые компилируются в единый исполняемый файл.

Таким образом, использование Microsoft Visual Studio обеспечивает высокую производительность труда разработчика и позволяет создать программный продукт с профессиональным уровнем архитектуры и интерфейса, минимизируя риски возникновения критических ошибок в процессе эксплуатации тренажера.

2.2.3 Работа с базой данных SQL

В рамках разработки приложения «Эрудит-тренажер» важным техническим аспектом является организация надежного и производительного хранилища лексических данных.

Для управления массивом слов, из которых система выбирает секретные комбинации и по которым проводит валидацию пользовательского ввода, была интегрирована база данных SQL.

Использование реляционной модели данных обусловлено

необходимостью быстрой выборки информации и обеспечения целостности словаря.

В контексте работы с Visual Studio и языком C# наиболее эффективным решением выступает использование локальной базы данных (например, SQLite или SQL Server Express), которая позволяет хранить тысячи существительных в структурированном виде, разделяя их по таким атрибутам, как идентификатор, само слово и его длина.

Техническое взаимодействие между программным кодом приложения и базой данных SQL реализуется с помощью технологии ADO.NET или объектно-реляционного отображения (Entity Framework).

Это позволяет приложению отправлять структурированные запросы к таблице слов, выполняя операцию случайной выборки записи для начала новой игровой сессии или проверку существования введенной пользователем последовательности символов в словаре.

Использование SQL-запросов значительно повышает эффективность работы с данными по сравнению с обычными текстовыми файлами, так как механизмы индексации в SQL позволяют проводить поиск слова практически мгновенно, что критически важно для сохранения динамики игрового процесса.

Кроме того, база данных обеспечивает удобство администрирования: при необходимости расширения словаря или добавления новых уровней сложности (например, слов разной длины) структура таблицы позволяет легко вносить изменения без существенной переработки программного кода.

Таким образом, использование технологий SQL в данном проекте гарантирует масштабируемость системы, высокую скорость обработки запросов и надежное хранение лингвистической базы тренажера.

2.2.4 Пользовательский интерфейс приложения

Проектирование и реализация пользовательского интерфейса приложения «Эрудит-тренажер» базируется на принципах интуитивности, минимализма и мгновенной визуальной обратной связи.

Основная цель графической оболочки — обеспечить максимально комфортную среду для решения лингвистических задач, не отвлекая пользователя лишними элементами управления.

Визуальная концепция интерфейса строится вокруг центрального игрового поля, представляющего собой матрицу ячеек, где каждая строка соответствует одной попытке угадывания.

Дизайн ячеек предусматривает динамическое изменение их состояния: программно реализованная смена цвета фона (серый, желтый, зеленый) после каждой проверки слова служит ключевым индикатором, позволяющим игроку логически вычислять правильную комбинацию букв.

Техническая реализация интерфейса в среде Visual Studio с использованием Windows Forms или WPF подразумевает создание событийной модели взаимодействия.

Важным элементом управления является виртуальная клавиатура, дублирующая раскладку символов.

Она не только обеспечивает возможность ввода данных с помощью манипулятора (мыши), но и выступает в роли дополнительного информационного табло, окрашивая клавиши уже использованных букв в соответствующие цвета.

Это значительно снижает когнитивную нагрузку на пользователя, избавляя его от необходимости запоминать статус каждой буквы алфавита. Также в интерфейс интегрированы управляющие компоненты для запуска новой игры, вывода справочной информации и отображения статистики текущей сессии.

					РПМ.340000.000 ПЗ	Лист.
						30
Изм.	Лист	№ докум.	Подп.	Дата		

Особое внимание уделено адаптивности и шрифтовому оформлению: выбранные кегли и начертания букв обеспечивают высокую читаемость на различных разрешениях экрана, а анимационные эффекты при переходе состояний ячеек делают процесс взаимодействия более плавным и современным.

Таким образом, пользовательский интерфейс приложения объединяет в себе эстетическую привлекательность и функциональную эффективность, создавая целостное пространство для интеллектуального тренинга.

2.2.5 Архитектура приложения

Архитектура приложения «Эрудит-тренажер» базируется на принципах модульности и разделения ответственности, что обеспечивает гибкость системы и упрощает процесс её отладки и сопровождения.

В основу программного решения заложена классическая многоуровневая структура, где каждый компонент отвечает за конкретный аспект функционирования игры.

На верхнем уровне располагается слой представления (Presentation Layer), реализованный с помощью графических инструментов .NET, который отвечает за визуализацию игрового процесса, захват пользовательского ввода и отображение динамических изменений состояний ячеек.

Архитектура показана на рисунке 9



Рисунок 9 – Архитектура

Центральным элементом системы является слой бизнес-логики (Business Logic Layer), где сосредоточены основные алгоритмы функционирования тренажера.

Здесь инкапсулированы методы выбора случайного слова, логика обработки игровых попыток и, что наиболее важно, алгоритм сравнения строк, который проводит посимвольный анализ вводимого слова на соответствие эталону.

Данный слой изолирован от интерфейсных решений, что позволяет при необходимости изменять визуальную составляющую без вмешательства в вычислительное ядро программы.

Логика управления состояниями игры (ожидание ввода, валидация, завершение сессии) также сосредоточена в этом блоке, обеспечивая строгое соблюдение правил игрового цикла.

Нижний уровень архитектуры представлен слоем доступа к данным (Data Access Layer), который отвечает за взаимодействие с внешними источниками информации.

В данном проекте этот слой обеспечивает связь с базой данных SQL или ресурсными файлами, содержащими лексическую основу. Модуль доступа к данным реализует методы загрузки словаря в оперативную память при старте приложения и выполнение поисковых запросов для мгновенной проверки существования вводимых слов.

Использование такой архитектурной модели позволяет минимизировать связность компонентов (low coupling) и повысить их сплоченность (high cohesion), что гарантирует стабильную работу «Эрудит-тренажера» и создает надежную базу для дальнейшей технической модернизации программного продукта.

2.4 Вывод

В рамках специального раздела была проведена детальная техническая проработка проекта «Эрудит-тренажер», позволившая трансфицировать концептуальную идею в структурированный план программной реализации.

Описание пользовательского сценария взаимодействия позволило четко зафиксировать логику поведения системы на каждом этапе — от инициализации игровой сессии и выбора секретного слова до обработки финальных событий победы или поражения.

Это обеспечило понимание динамики приложения и сформировало требования к его функциональным модулям.

Обоснование и детальное описание инструментальных средств разработки подтвердили целесообразность использования стека технологий Microsoft Visual Studio и языка программирования C#.

Технические возможности выбранной среды, дополненные мощными механизмами обработки строковых данных и интеграцией с базой данных SQL, формируют надежный фундамент для создания производительного и отказоустойчивого ПО.

Спроектированная многоуровневая архитектура приложения гарантирует четкое разделение ответственности между интерфейсом, бизнес-логикой и слоем данных, что значительно упрощает дальнейшую отладку и масштабирование системы.

Таким образом, по итогам раздела была полностью сформирована техническая база, необходимая для перехода к практическому этапу написания кода и тестирования.

3 Программная реализация

В рамках программной реализации ключевого модуля приложения «Эрудит-тренажер» была разработана процедура обработки игрового шага, которая функционирует на основе алгоритма посимвольного сравнения двух строковых массивов, представляющих собой введенное пользователем слово и эталонное значение, выбранное системой из базы данных.

При этом логика работы программы организована таким образом, что на первом этапе выполняется цикл первичной валидации для выявления полных совпадений символов, находящихся на идентичных индексных позициях.

В результате чего соответствующим графическим объектам игрового поля присваивается статус подтвержденного попадания и активируется перерисовка фона ячеек в зеленый цвет.

После завершения первичного цикла система переходит к этапу вторичного анализа оставшихся символов для выявления их наличия в составе загаданного слова без учета конкретной позиции, что реализуется через проверку вхождений в динамический список оставшихся букв и приводит к окрашиванию ячеек в желтый цвет, сигнализирующий о необходимости перемещения буквы в следующей попытке.

В то время как все не прошедшие данные фильтры символы автоматически помечаются как отсутствующие в лексеме и получают серое цветовое оформление на системном уровне.

Данная последовательность действий сопровождается строгим контролем количества повторений каждой буквы для исключения ложных срабатываний индикации в словах с дублирующимися знаками, что гарантирует математическую точность подсказок и обеспечивает пользователю корректную аналитическую базу для построения дальнейшей стратегии угадывания в соответствии с классическими правилами механики лингвистических головоломок, реализованных средствами языка

программирования высокого уровня в среде визуального проектирования интерфейсов.

3.1 Реализация интерфейса пользователя

Процесс реализации интерфейса в среде разработки Visual Studio базировался на использовании технологии Windows Forms или WPF что позволило создать многослойную визуальную композицию объединяющую интерактивные элементы управления и статическую сетку игрового поля для ввода букв при этом на программном уровне каждое знакоместо было представлено в виде отдельного графического контейнера или текстового блока обладающего набором динамически изменяемых свойств включая цвет фона толщину рамки и параметры шрифта что обеспечивает мгновенную визуальную трансформацию ячейки при получении сигнала от алгоритма проверки слова.

Кратко показана структура формы на листинге №1

Листинг №1

```
public static class Session
{
    public static int UserId { get; set; }
    public static string Login { get; set; } = "";
    public static string FullName { get; set; } = "";
    public static bool IsAdmin { get; set; }

    public static void Clear()
    {
        UserId = 0;
        Login = "";
        FullName = "";
        IsAdmin = false;
    }
}
```


Для того чтобы получить доступ к функционалу программного продукта, пользователю необходимо авторизоваться путём введения логина и пароля №2

Листинг 2 – Авторизация пользователя

```
public class AccountsForm : Form
{
    private readonly DataGridView _grid = new()
    {
        Dock = DockStyle.Fill,
        ReadOnly = true,
        SelectionMode = DataGridViewSelectionMode.FullRowSelect,
        MultiSelect = false,
        AllowUserToAddRows = false,
        AutoSizeColumnsMode =
DataGridViewAutoSizeColumnsMode.Fill,
        BorderStyle = BorderStyle.FixedSingle
    };
    private readonly TextBox _login = new();
    private readonly TextBox _fullName = new();
    private readonly TextBox _email = new();
    private readonly TextBox _phone = new();
    private readonly TextBox _notes = new();
    private readonly TextBox _newUserPass = new()
    {
        PlaceholderText = "Пароль: обязателен при «Добавить»; при
«Сохранить» – только если меняете",
        UseSystemPasswordChar = true
    };
    private readonly CheckBox _admin = new() { Text =
"Администратор", AutoSize = true };
    private readonly CheckBox _locked = new() { Text =
"Заблокирован", AutoSize = true };

    public AccountsForm()
```

```

        {
            if (!Session.IsAdmin)
            {
                MessageBox.Show("Доступ только для администратора.",
Text, MessageBoxButtons.OK, MessageBoxIcon.Warning);
                Shown += (_, _) => BeginInvoke(new Action(Close));
            }

            Text = "Учётные записи игроков";
            ClientSize = new Size(1080, 640);
            StartPosition = FormStartPosition.CenterParent;
            BackColor = GameTheme.BgDeep;
            Font = GameTheme.UiFont(10F);

            GameTheme.StyleDataGrid(_grid);

            foreach (var t in new[] { _login, _fullName, _email,
_phone, _notes, _newUserPass })
            {
                t.BackColor = GameTheme.BgPanel;
                t.ForeColor = GameTheme.TextPrimary;
                t.BorderStyle = BorderStyle.FixedSingle;
            }
        }

```

Центральная часть интерфейса организована в виде матрицы из шести строк по пять столбцов, где каждая строка активируется последовательно по мере расходования игровых попыток.

А ниже основного поля размещена функциональная виртуальная клавиатура, которая выступает не только инструментом ввода данных для пользователей, не имеющих доступа к физической клавиатуре, но и выполняет роль вспомогательного информационного табло, синхронизируя свою цветовую палитру с результатами анализа вводимых слов.

Для реализации отзывчивости интерфейса были разработаны обработчики событий нажатия клавиш, которые контролируют корректность заполнения текущей строки и предотвращают переход к следующему этапу до

момента полной валидации пятибуквенной комбинации.

В дополнение к основной игровой зоне в интерфейс интегрированы служебные компоненты, такие как модальные окна для вывода итоговой статистики, кнопка сброса текущей сессии для генерации нового секретного слова и текстовые индикаторы состояния системы, оповещающие об ошибках ввода или отсутствии слова в базе данных.

Совокупность данных решений позволила создать целостное и эргономичное рабочее пространство, обеспечивающее высокую степень вовлеченности пользователя и минимизирующее время на освоение правил взаимодействия с программным продуктом.

3.2 Разработка программного продукта

Непосредственная разработка программного продукта в среде Visual Studio началась с создания базового каркаса приложения и настройки модульной структуры проекта, что позволило изолировать графическую оболочку от вычислительной логики и обеспечить удобство при написании кода на языке C#.

В процессе реализации была сформирована основная форма приложения, выступающая в роли контейнера для всех визуальных компонентов, а также разработан вспомогательный класс обработки данных, отвечающий за взаимодействие с внешними ресурсами и хранение текущего игрового состояния.

Программный код был разделен на несколько функциональных блоков, среди которых ключевое место занимает процедура инициализации, выполняемая при каждом запуске новой сессии и включающая в себя обращение к базе данных SQL для получения случайного идентификатора слова с последующей загрузкой его символьного представления в оперативную память.

Для обеспечения динамики игрового процесса была реализована сложная система событийных фильтров, которая перехватывает нажатия клавиш на клавиатуре, распределяет вводимые символы по соответствующим ячейкам текущей строки и блокирует возможность ввода при достижении лимита в пять знаков до момента нажатия клавиши подтверждения.

Алгоритмическая часть разработки была сосредоточена на создании высокоэффективного метода сравнения строк, который работает в два прохода, гарантируя корректное определение позиционных совпадений и исключая дублирование подсказок для повторяющихся букв.

А завершающим этапом создания продукта стала интеграция системы визуального фидбека, которая связывает логические результаты проверки с изменением свойств графических примитивов на экране.

Таким образом, в ходе разработки была построена устойчивая программная экосистема, способная обрабатывать пользовательские сценарии любой сложности, сохраняя при этом высокую скорость отклика интерфейса и точность выполнения математических операций внутри игрового цикла, что делает итоговый тренажер надежным инструментом для развития лингвистических способностей.

Заключение

В ходе выполнения курсовой работы была рассмотрена задача разработки интеллектуального приложения «Эрудит-тренажер», актуальная в условиях растущего интереса к геймификации образовательных процессов и цифровым методам развития когнитивных способностей.

В процессе работы был проведен комплексный анализ предметной области, позволивший выявить ключевые механики функционирования лингвистических игр и формализовать требования к современным программным тренажерам для расширения словарного запаса.

На основе проведенного анализа была сформулирована цель разработки и определен перечень задач, необходимых для создания стабильного приложения в среде Visual Studio. Особое внимание было уделено проектированию пользовательского сценария и алгоритма цветовой индикации, так как точность обратной связи и интуитивность интерфейса являются фундаментальными факторами для вовлечения игрока и эффективности обучающего процесса.

Разработка приложения «Эрудит-тренажер» позволяет автоматизировать процесс проверки лингвистических гипотез пользователя, обеспечивая мгновенный анализ введенных данных и визуализацию результатов с помощью классической механики цветowych подсказок.

Использование такой системы способствует развитию логического мышления, улучшению памяти и повышению общей грамотности за счет работы с верифицированной базой слов на основе SQL-технологий.

Полученные в ходе выполнения курсовой работы результаты могут быть использованы как основа для создания более сложных обучающих платформ или интегрированы в образовательные системы в качестве вспомогательного инструмента.

Кроме того, данная работа способствовала формированию практических

					<i>РПМ.340000.000 ПЗ</i>	Лист.
						40
Изм.	Лист	№ докум.	Подп.	Дата		

навыков проектирования многоуровневой архитектуры программного обеспечения, реализации алгоритмов сравнения строк на языке C# и работы с реляционными данными.

Таким образом, поставленная цель курсовой работы была достигнута, а полученные выводы подтверждают техническую целесообразность и практическую значимость разработки интеллектуального приложения «Эрудит-тренажер».

					РПМ.340000.000 ПЗ	Лист.
						41
Изм.	Лист	№ докум.	Подп.	Дата		

Перечень использованных информационных ресурсов

1. Мезенцев, К. Н. Автоматизированные информационные системы. М.: Академия, 2021.
2. Сергиевский, Г. М. Функциональное и логическое программирование, 2022.
3. Соколова, В.В. Вычислительная техника и информационные технологии. Разработка приложений. Учебное пособие для прикладного бакалавриата / В.В. Соколова. - М.: Юрайт, 2022.
4. Балашов, Е. П. Проектирование информационно-управляющих систем / Е.П. Балашов, Д.В. Пузанков. - М.: Радио и связь, 2021.
5. Боковой, Ю. В. Особенности методологии проектирования информационных систем для малого и среднего бизнеса / Ю.В. Боковой. - М.: Синергия, 2022.
6. Мельников В. В., Мерзлякова Н. И., Берестнев Д. В. Информационные системы и технологии: Учебник для вузов. — Юрайт, 2023.
7. Хилл Г., Флойд К. Цифровая бизнес-трансформация: создание ценности в эпоху цифровой экономики. — Питер, 2021.
8. Кузьмичев В. В., Кочегурова Т. В. Информационные системы в управлении организацией: Учебник. — Юрайт, 2023.
9. Голощапов В. Н., Политов В. Е., Шалаева Н. В. Организация информационных систем: Учебник. — Юрайт, 2023.
10. Габрельян О. Г. Информационные системы и базы данных: Учебник для вузов. — Издательство Юрайт, 2021.
11. Кудрявцев Ю. В., Николаев И. Ю. Базы данных. Курс лекций. – БХВ-Петербург, 2022.
12. Кудрявцев Ю. В., Николаев И. Ю. Проектирование баз данных. Учебник. – БХВ-Петербург, 2022.